CHAPTER 3

Binary Arithmetic & Complements

Add · subtract · multiply · divide in binary · 2's complement tricks · overflow

QUICK TRICK

This chapter has many QUICK TRICKS (green boxes). Learn them. They save time in the exam. A summary of all tricks is at the end of the chapter.

3.1 Binary addition

● **LEVEL 1 · BEGINNER**

Idea. Binary addition is like decimal addition. But we only have digits 0 and 1. So when a column adds up to 2 or more, we write the small part and **carry** 1 to the next column.

Adding	Result	Write	Carry
0 + 0	0	0	0
0 + 1	1	1	0
1 + 0	1	1	0
1 + 1	2 (binary 10)	0	1
1 + 1 + 1 (with a carry)	3 (binary 11)	1	1

QUICK TRICK

Count-the-ones trick. In each column, count how many 1s you have (including the carry). **Write 0 if the count is even, write 1 if the count is odd. Carry 1 if the count is 2 or 3** (carry 0 if the count is 0 or 1).
Count 0 → write 0, carry 0 | Count 1 → write 1, carry 0 | Count 2 → write 0, carry 1 | Count 3 → write 1, carry 1.

Example: 1011 + 0110 (11 + 6)

```
carry : 1 1 1 . .
A     :   1 0 1 1        (11)
B     : + 0 1 1 0        ( 6)
        ----------
sum   : 1 0 0 0 1        (17)
```

Go from right to left. Column 1: 1+0 = 1. Column 2: 1+1 = 2 → write 0, carry 1. Column 3: 0+1+carry 1 = 2 → write 0, carry 1. Column 4: 1+0+carry 1 = 2 → write 0, carry 1. The last carry becomes the leftmost 1.

QUICK TRICK

Always check by converting to decimal. 1011 = 11, 0110 = 6, 10001 = 17. And 11 + 6 = 17. ✔

3.2 Binary subtraction

● LEVEL 1 · BEGINNER

Subtracting	Result	Note
0 − 0	0	
1 − 0	1	
1 − 1	0	
0 − 1	1 and **borrow 1**	You cannot take 1 from 0, so borrow from the next column

How to borrow. Borrowing from the next column on the left gives you **2** in the current column (binary 10). So 0 − 1 becomes 2 − 1 = 1. The column you borrowed from loses 1.

Example: 1001 − 0101 (9 − 5)

```
Column 1 (right): 1 - 1 = 0
Column 2        : 0 - 0 = 0
Column 3        : 0 - 1 -> borrow from column 4 -> 2 - 1 = 1   (column 4 becomes 0)
Column 4        : 0 - 0 = 0
Answer          : 0100   (= 4)
```

COMMON MISTAKE
Borrowing is confusing and easy to get wrong in the exam. That is why computers (and smart students) use the **2's complement method** in the next sections. It turns subtraction into addition.

3.3 Complements — a fast way to make negative numbers

● LEVEL 2 · EXAM

Why do we need this? Computers can add. They are bad at borrowing. A **complement** is a trick to turn a subtraction into an addition. There are two types.

1's complement — “Flip all the bits”

```
0 becomes 1,  1 becomes 0
1's complement of 101100  =  010011
```

2's complement — “1's complement, then add 1”

```
2's complement of 101100:
    step 1: flip all bits  ->  010011
    step 2: add 1          ->  010011 + 1 = 010100
```

QUICK TRICK
SCAN-FROM-RIGHT TRICK (no need to add 1!)
Start at the right. **Copy the bits until you copy the first 1 (include it). Then flip all the remaining bits.**
Example: 101**1**00 → from the right: 0, 0, then first 1. Copy **100**. Flip the rest (101) → 010. Answer: **010 100**.
Another: 0110**1**0 → copy **10**, flip 0110 → 1001 → **100110**.

Number	2's complement (scan trick)	Reason it works
0110 (6)	1010	copy '10', flip '01' → '10'
1000 (−8)	1000	copy '1000' (first 1 is the leftmost bit), nothing left to flip
0000	0000	no 1 found: everything is copied (0 has no negative)
0001 (1)	1111	copy '1', flip '000' → '111'

QUICK TRICK

Two more tricks with complements

1) **Double negative:** the 2's complement of a 2's complement gives back the original number.

2) **Formula:** 2's complement of x = $2^n - x$ (n = number of bits). Quick way to read a negative 8-bit number: 11101111 is 239 as unsigned. 256 − 239 = 17. So it is **−17**.

EASY MEMORY

1's complement = **flip**. 2's complement = **flip + 1** = the **scan trick**. Negative of a number = its 2's complement.

3.4 Subtraction using 2's complement

● **LEVEL 2 · EXAM**

Idea. A − B is the same as A + (−B). And −B is the 2's complement of B. So we only need an adder.

```
STEPS
1) Write A and B with the SAME number of bits.
2) Find the 2's complement of B (use the scan trick).
3) Add A + (2's complement of B).
4) Look at the CARRY coming out of the leftmost bit:
     carry = 1  ->  answer is POSITIVE. Throw the carry away.
                    What is left is the answer.
     carry = 0  ->  answer is NEGATIVE. What is left is in 2's complement form.
                    Take its 2's complement to get the size of the answer.
```

QUICK TRICK

Carry trick: "Carry 1 = Positive". If a carry comes out, the answer is positive (or zero). If no carry, the answer is negative. *(This rule is for subtraction with 2's complement.)*

Example 1: 9 − 5 (4 bits) → carry = 1, positive

A = 1001, B = 0101. 2's complement of B (scan trick): copy '1', flip '010' → **1011**.

Add: 1001 + 1011 = **1** 0100. Carry = 1 → positive. Throw the carry away → **0100 = 4**. ✔

Example 2: 5 − 9 (4 bits) → carry = 0, negative

A = 0101, B = 1001. 2's complement of B: copy '1', flip '100' → **0111**.

Add: 0101 + 0111 = 1100. No carry → negative.

Size of the answer: 2's complement of 1100 = copy '100', flip '1' → '0' → 0100 = 4. So the answer is **−4**. ✔

Example 3 (Level 2): 45 − 72 (8 bits)

45 = 00101101, 72 = 01001000.

2's complement of 72: copy '1000', flip '0100' → '1011' → **10111000**.

Add: 00101101 + 10111000 = 11100101. No carry → negative.

Size: 2's complement of 11100101 = copy '1', flip '1110010' → '0001101' → 00011011 = 27. Answer **−27**. ✔

Subtraction using 1's complement (older method, sometimes asked)

```
1) Add A + (1's complement of B).
2) If there is a carry out: ADD that carry back to the result (this is called
   END-AROUND CARRY). The answer is positive.
3) If there is no carry: the answer is negative. Flip all bits of the result
   to get its size.
```

Example: 9 − 5 and 5 − 9 (1's complement, 4 bits)

9 − 5: 1001 + 1010 (1's comp of 0101) = 1 0011. Carry 1 → add it back: 0011 + 1 = **0100 = 4**. ✔

5 − 9: 0101 + 0110 (1's comp of 1001) = 1011. No carry → negative. Flip 1011 → 0100 = 4 → **−4**. ✔

EXAM TRAP

1's complement needs the **end-around carry**. **2's complement** does NOT: it just throws the carry away.

3.5 Binary multiplication

● **LEVEL 1 · BEGINNER**

Idea. Same as decimal "long multiplication", but much easier: each digit of the bottom number is only 0 or 1.

```
Rule : bottom bit = 1  -> write the top number
       bottom bit = 0  -> write all zeros
Move one place to the LEFT for every new bottom bit.  Then ADD all the rows.
```

Example: 101 × 011 (5 × 3)

```
        1 0 1        (5)
      x 0 1 1        (3)
      ---------
        1 0 1        <- bottom bit 1 : copy 101
      1 0 1 0        <- bottom bit 1 : copy 101, shifted 1 place left
    0 0 0 0 0        <- bottom bit 0 : zeros, shifted 2 places
      ---------
      0 1 1 1 1      (15)
```

QUICK TRICK

Shift trick. Multiplying by 2 = shift left by 1 place (add a 0 on the right). Multiplying by 4 = shift left 2 places. Multiplying by 2^k = shift left k places.
Example: 1011 (11) × 4 = 101100 (44).

GATE POINT

Multiplying an n-bit number by an m-bit number gives a result of up to **n + m bits**. Two 8-bit numbers → up to 16 bits.

3.6 Binary division

● LEVEL 2 · EXAM

Idea. Same as decimal long division. At each step you check: does the divisor fit? If yes, write 1 in the quotient and subtract. If no, write 0.

Example: 1101 ÷ 11 (13 ÷ 3)

Take the first 1: 1 is smaller than 11 → quotient bit **0** (skip this leading zero in the answer).

Take 11: 11 fits in 11 → quotient bit **1**, subtract: 11 − 11 = 0.

Bring down 0 → 0. 0 is smaller than 11 → quotient bit **0**.

Bring down 1 → 01. 01 is smaller than 11 → quotient bit **0**.

Quotient = **100** (= 4), remainder = **1**. Check: 3 × 4 + 1 = 13. ✔

QUICK TRICK

Divide-by-2 trick. Dividing by 2 = shift right by 1 place. The bit that falls off the right end is the **remainder**. Example: 1101 (13) ÷ 2 → 110 (6), remainder 1.

3.7 Signed arithmetic with 2's complement

● LEVEL 2 · EXAM

Great news. In 2's complement, you just **add** the numbers, positive or negative. The sign takes care of itself. **Throw away any carry that comes out of the leftmost bit.**

Examples (4-bit, range −8 to +7)

(+5) + (−3): 0101 + 1101 = 1 0010 → drop carry → **0010 = +2**. ✔

(−5) + (−3): 1011 + 1101 = 1 1000 → drop carry → **1000 = −8**. ✔ (fits in 4 bits)

(−6) + (−5): 1010 + 1011 = 1 0101 → drop carry → 0101 = +5. ✘ **Wrong!** The real answer is −11, which does not fit in 4 bits. This is **overflow** (next section).

EASY MEMORY

In 2's complement: **ADD, drop the carry, and check overflow.** Nothing else.

3.8 Overflow

● LEVEL 3 · GATE

What is overflow? The correct answer is too big (or too small) to fit in the number of bits we have. The computer then shows a wrong answer. Think of a glass that is too small for the water.

Unsigned numbers

Overflow happens when there is a carry out of the leftmost bit. Example (4-bit): 1100 + 0111 = 1 0011. Carry out = 1 → overflow. (12 + 7 = 19 does not fit in 4 bits, max is 15.)

Signed numbers (2's complement)

You add	Overflow possible?	How to know
Positive + Positive	Yes	Overflow if the result looks **negative** (sign bit 1)
Negative + Negative	Yes	Overflow if the result looks **positive** (sign bit 0)
Positive + Negative	**Never**	The result always fits

QUICK TRICK

Sign-check trick. Overflow only if the two numbers you add have the **same sign** and the answer has the **opposite sign**.

Second method: overflow = (carry INTO the leftmost bit) XOR (carry OUT of the leftmost bit). If they are different → overflow.

Third method (fastest): work out the true answer in decimal. If it is outside -2^{n-1} to $2^{n-1} - 1$, there is overflow.

Example: 0110 + 0101 (6 + 5), 4-bit signed

Add: 0110 + 0101 = 1011. Two positives gave a negative sign bit → **overflow**.

Carry check: carry into leftmost bit = 1, carry out = 0. 1 XOR 0 = 1 → overflow. ✔

Decimal check: 6 + 5 = 11 is bigger than 7 → overflow. ✔

Example: 1111 + 0001 (−1 + 1)

Add: 1111 + 0001 = **1** 0000. Carry out = 1. Carry into leftmost bit = 1. 1 XOR 1 = 0 → **no overflow**. Answer 0000 = 0. ✔

EXAM TRAP

A carry out does NOT always mean overflow for signed numbers. Carry out means overflow only for **unsigned** numbers.

Overflow in subtraction

A − B means A + (−B). So overflow can happen only when A and B have **different signs** (because then A and −B have the same sign).

QUICK TRICK

Subtraction overflow trick: A − B overflows if A and B have **different signs** and the result has the **same sign as B** (different from A). Example: 6 − (−6): A = 0110, B = 1010, 2's comp of B = 0110, sum = 1100. A is +, B is −, result is − → overflow (12 does not fit).

COMMON MISTAKE

Forgetting to make both numbers the **same length** before adding. Use sign extension (Chapter 2) first: copy the leftmost bit to make the shorter number longer.

Quick-Solve Tricks — Summary

Situation	Trick
Add binary numbers	Count the 1s in the column. Even → write 0. Odd → write 1. Count 2 or 3 → carry 1.
Find 2's complement	**Scan from the right: copy up to and including the first 1, then flip the rest.**
Find 1's complement	Flip every bit.
Read a negative 2's complement number	Take its 2's complement to get the size, then put a minus sign. Or: 2^n − (unsigned value).
Subtract A − B	A + 2's complement of B. **Carry 1 = positive** (drop it). No carry = negative.
1's complement subtraction	Add. If carry, add it back (end-around carry).
Multiply / divide by 2^k	Shift left / right by k places. (Right shift: the dropped bits are the remainder.)
Signed overflow	Same signs in, opposite sign out. + and − never overflow.
Unsigned overflow	Carry out of the leftmost bit.
Any answer	Check by converting to decimal.

GATE QUICK CHECK — Chapter 3

```
n-bit unsigned        : 0 .. 2^n - 1
n-bit 2's complement  : -2^(n-1) .. 2^(n-1) - 1
2's complement        : flip + 1  (or scan trick)
Subtraction           : A - B = A + (2's comp of B)
Signed overflow       : C_in(MSB) XOR C_out(MSB) = 1
Unsigned overflow     : C_out(MSB) = 1
n-bit + n-bit         : result may need n + 1 bits
n-bit x n-bit         : result may need 2n bits
```

Traps: carry out is not always overflow · 1's complement needs end-around carry · the numbers must have equal length · the most negative number has no positive partner · positive + negative never overflows.

Practice questions

● **LEVEL 1 · BEGINNER**

Q1. Add 1010 + 0111.

Q2. Subtract 1100 − 0101 (use any method).

Q3. Find the 2's complement of 0011 (use the scan trick).

● **LEVEL 2 · EXAM**

Q4. Using 8-bit 2's complement, compute 45 − 72.

Q5. Multiply 1101 × 101.

Q6. 4-bit signed numbers: add 0101 + 0100. Is there overflow?

● **LEVEL 3 · GATE**

Q7. Two 8-bit unsigned numbers are added. How many bits are needed to be sure the answer always fits?

Q8. 8-bit signed: add 10000000 + 11111111. Give the carry into and out of the leftmost bit. Is there overflow?

Q9. 4-bit signed: compute 0110 − 1010. Is there overflow?

Q10. The 8-bit 2's complement of a number x is 10110100. Find x.

Q11. In 8-bit 2's complement, what happens when we find the negative of 10000000?

Q12. In 4-bit 1's complement, add 0110 + 1010 (that is 6 + (−5)).

Answers

Q	Solution
1	1010 + 0111 = **10001** (10 + 7 = 17). Columns: 0+1=1; 1+1=0 carry 1; 0+1+1=0 carry 1; 1+0+1=0 carry 1; last carry 1.
2	2's complement of 0101: copy '1', flip '010' → 1011. 1100 + 1011 = 1 0111. Carry 1 → positive → **0111** (12 − 5 = 7).
3	0011: copy '1', flip '001' → '110'. Answer **1101** (= −3).
4	**Given** 45, 72. **Concept** A + 2's comp of B. **Calculation** 72 = 01001000 → 2's comp = 10111000. 00101101 + 10111000 = 11100101, no carry → negative. Size: 2's comp of 11100101 = 00011011 = 27. **Answer −27**.
5	1101 (13) × 101 (5): row 1 = 1101, row 2 = 000000 (shifted 1, bit 0), row 3 = 110100 (shifted 2). 001101 + 110100 = **1000001** = 65. ✔
6	0101 + 0100 = 1001. Two positives gave a negative sign bit → **overflow** (5 + 4 = 9 > 7).
7	Largest sum: 255 + 255 = 510. 2^9 = 512 > 510, so **9 bits** (n + 1).
8	10000000 + 11111111: the lower 7 bits are 0000000 + 1111111 = 1111111, no carry, so **carry into leftmost bit = 0**. Leftmost bits 1 + 1 = 10 → **carry out = 1**. 0 XOR 1 = 1 → **overflow**. (−128 + −1 = −129, too small.) The 8-bit result 01111111 = +127 is wrong.
9	6 − (−6) = 12. A is +, B is − (different signs). 2's comp of 1010 = 0110. 0110 + 0110 = 1100 (looks negative). Result sign is different from A → **overflow** (12 > 7).
10	Double-negative trick: take the 2's complement of 10110100. Scan: copy '100', flip '10110' → '01001'. Answer **01001100 = 76**. Check: 2's comp of 76 = 10110100 ✔.
11	10000000 = −128. Its 2's complement is 10000000 again (scan trick: copy '10000000', nothing left to flip). +128 does not fit in 8 bits, so the negative of −128 **cannot be represented; overflow**.
12	0110 + 1010 = 1 0000. Carry 1 → add it back (end-around): 0000 + 1 = **0001 = +1**. (6 − 5 = 1) ✔

Next: Chapter 4 — Boolean Algebra (laws, De Morgan, consensus, duality, step-by-step simplification, with simple tricks).